

---

## Table of Contents

|                                                                                |    |
|--------------------------------------------------------------------------------|----|
| .....                                                                          | 1  |
| signal d'entree .....                                                          | 1  |
| determiner la reponse impulsionnelle $h[n]$ obtenue par l'equation (1.1) ..... | 2  |
| Affichage du signal déconvolué .....                                           | 9  |
| Calcul de l'énergie de l'erreur de reconstruction .....                        | 10 |
| Tracé de l'énergie de l'erreur en fonction de $\alpha$ .....                   | 13 |
| Recherche de la valeur de $\alpha$ qui minimise l'énergie de l'erreur .....    | 14 |

%Sujet du TP :

%3. Résolution d'un problème inverse : Déconvolution d'un signal triangulaire

%OUCHABANE AHLEM

%Avec une durée d'observation  $L_x = 100s$  et aussi utiliser une instruction du type  $x = [x_1 \ x_2]$  représenter

%2 tronçons successifs du signal.

%Le code suivant nous donne le résultat souhaité.

## signal d'entree

```
Lx = 100;
```

```
t1 = 0:Lx/2 - 1;
```

```
x1 = t1 / (Lx/2);
```

```
t2 = Lx/2:Lx - 1;
```

```
x2 = (-t2 + Lx) / (Lx/2);
```

```
tx=[t1 t2];
```

```
x = [x1 x2];
```

```
figure(1);
```

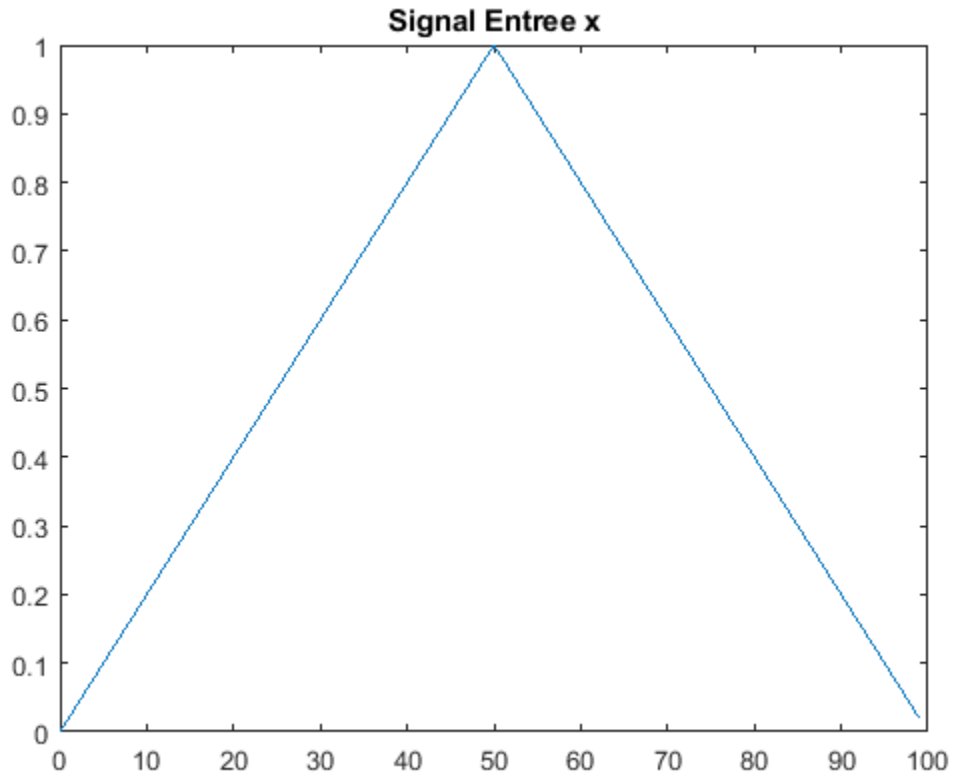
```
plot(tx, x);
```

```
title('Signal Entree x');
```

```
m= 15;
```

```
sigmah = 4;
```

```
t = 0:30;
```



**determiner la reponse impulsionnelle  $h[n]$  obtenue par l'equation (1.1)**

```
h_n = (1/(sigmah*sqrt(2*pi))) * exp((- (t-m).^2) / (2*sigmah.^2));
```

```
figure(2);  
plot(t, h_n);  
title('Reponse Impulsionnelle h[n]');
```

```
ynb_n = conv(x, h_n);
```

```
figure(3);  
plot(ynb_n);  
title('Signal de sortie non bruite ynb[n]');
```

```
y = round(ynb_n * 2^10) / 2^10;
```

```
figure(4);
```

---

```

stairs(y);

title('Signal Quantifié y');

b_n = y - ynb_n;

figure(5);

plot(b_n);

title('le bruit de quantification b[n]');

Ny=size(ynb_n,2);
Nx=size(x,2);
Nh=size(h_n,2);
display(Ny);
display(Nx);
display(Nh);

H_ligne = zeros(1,Nx);
H_colones = zeros(Ny,1);
H= toeplitz(H_colones,H_ligne);

    for i=1:Nx
        for j=1:31
            var = i + j - 1;
            H(var,i)=h_n(j);
        end
    end

% Pour adapter la dimension de H faut transposer la matrice x
x_t = transpose(x);

ynb_om = H * x_t;

t_om = 0:129;

figure(6);

plot(t_om, ynb_om);

title('Sortie non bruitée par une operation matricielle');
%Pour savoir l'erreur de troncature : Erreur = y - ynb

% Pour adapter la dimension de ynb_n faut transposer la matrice ynb_om
Erreur_t = ynb_om' - ynb_n;

figure(7);

plot(t_om, Erreur_t);

```

---

---

```

title('Erreur de troncature');

%La définition de deconv.
%[q, r] = deconv (u, v) déconvolution d'un vecteur v d'un vecteur u en
utilisant une division longue et
%renvoie le quotient q et le reste r tels que u = conv (v, q) + r.
[x_q,b_r]=deconv(y_n,h_n);
%Reconstruire le signal d'entrée x[n] et aussi le bruit b[n] avec la sortie
non bruitée y_n.
[x_rec,b_rec]=deconv(y,h_n);

%Reconstruire le signal d'entrée x[n] et aussi le bruit b[n] avec le signal
quantifié y.
%p. 6
%Et puis pour calculer le RSB du signal de sortie avec le signal reconstruit
x_rec et le bruit reconstruit
b_rec, et le résultat de RSB égale :

% Puissance du signal x_rec
Ps = sum(sum((x_rec - mean(mean(x_rec)))).^2));

% Puissance du bruit b_rec
Pn = sum(sum((b_rec).^2));

% Rapport signal sur bruit (SNR) en dB
snr = 10*log10(Ps/Pn);
%valeur du SNR
display(snr);

sigmah1 = 2;
h_n1 = (1/(sigmah1*sqrt(2*pi)))*(exp(-(t-m).^2)/(2*sigmah1^2)));
figure(8);
subplot(411);
plot(t, h_n1);
title('Reponse Impulsionnelle h[n] avec  $\sigma_h = 2$ ');

sigmah2 = 4;
h_n2 = (1/(sigmah2*sqrt(2*pi)))*(exp(-(t-m).^2)/(2*sigmah2^2)));
subplot(412);
plot(t, h_n2);
title('Reponse Impulsionnelle h[n] avec  $\sigma_h = 4$ ');

sigmah3 = 6;
h_n3 = (1/(sigmah3*sqrt(2*pi)))*(exp(-(t-m).^2)/(2*sigmah3^2)));
subplot(413);
plot(t, h_n3);
title('Reponse Impulsionnelle h[n] avec  $\sigma_h = 6$ ');

sigmah4 = 8;
h_n4 = (1/(sigmah4*sqrt(2*pi)))*(exp(-(t-m).^2)/(2*sigmah4^2)));
subplot(414);

```

---

---

```

plot(t, h_n4);
title('Reponse Impulsionnelle h[n] avec  $\sigma_h = 8$ ');

%Dans les résultats avant, on peut bien voir que si la valeur du  $\sigma_h$  été
bien augmentée. La durée de la
%
% réponse impulsionnelle doit devient plus grande. Donc il faut bien définir
la valeur de  $\sigma_h$ .

% Transposition de y pour assurer la compatibilité des dimensions
Y = y';

% Calcul de la solution des moindres carrés
x_mc = inv(H'*H) * H' * Y;

Ny =

    130

Nx =

    100

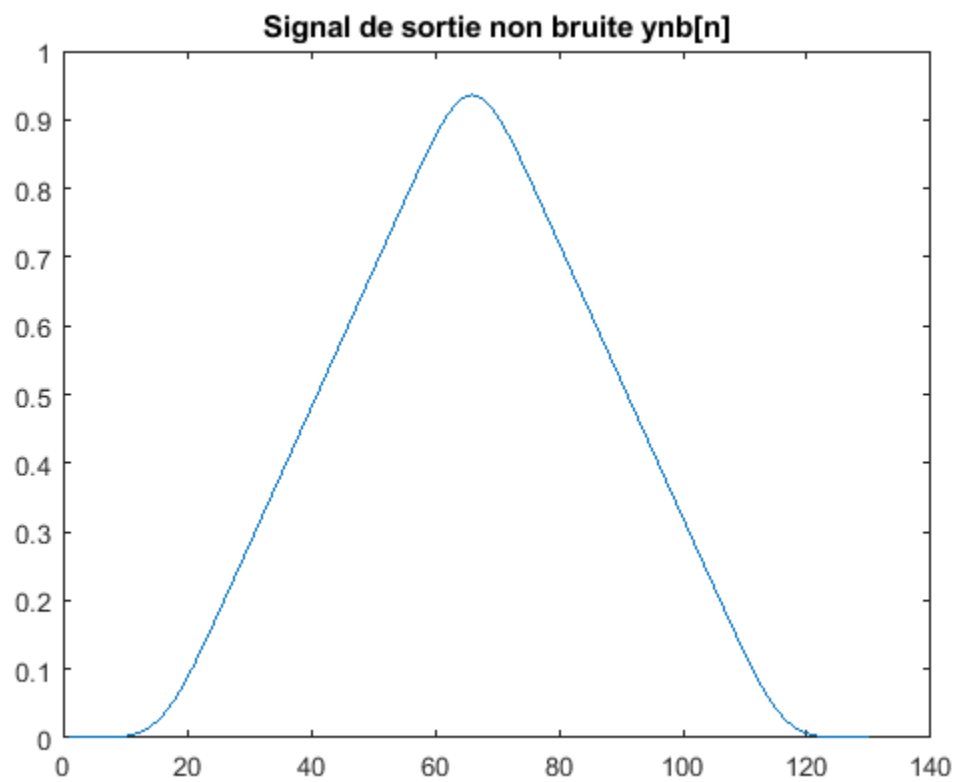
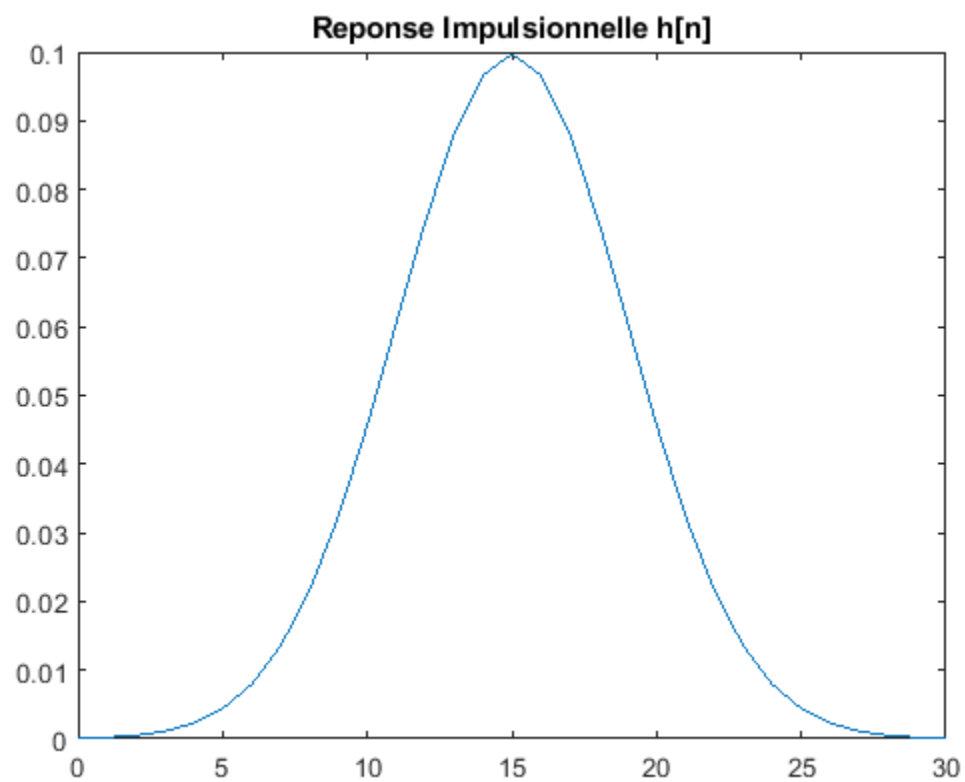
Nh =

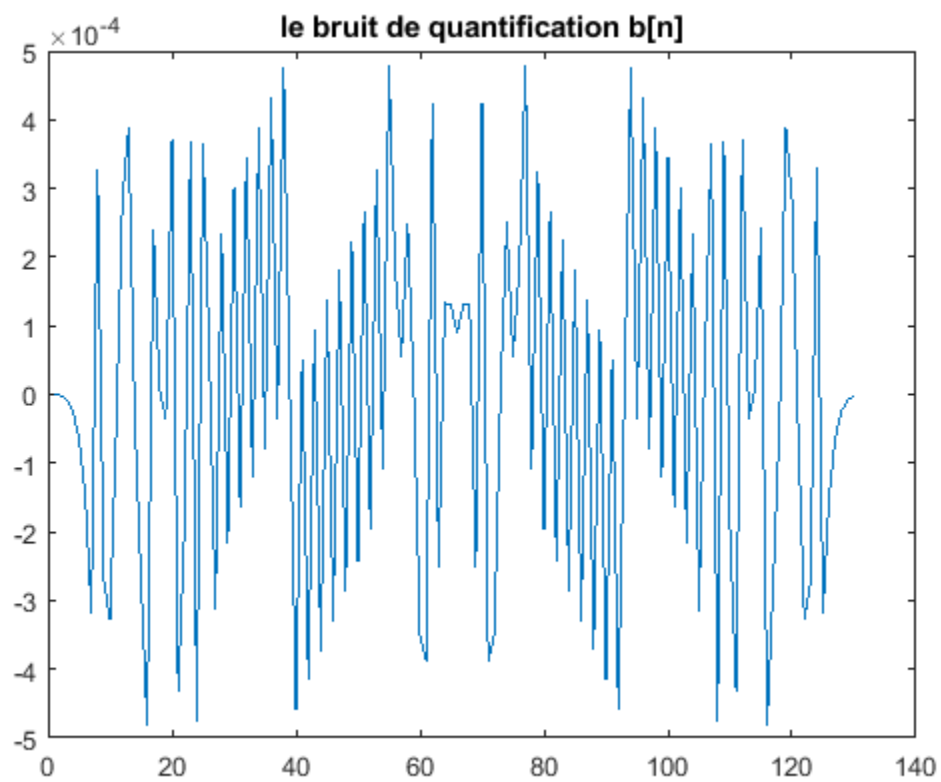
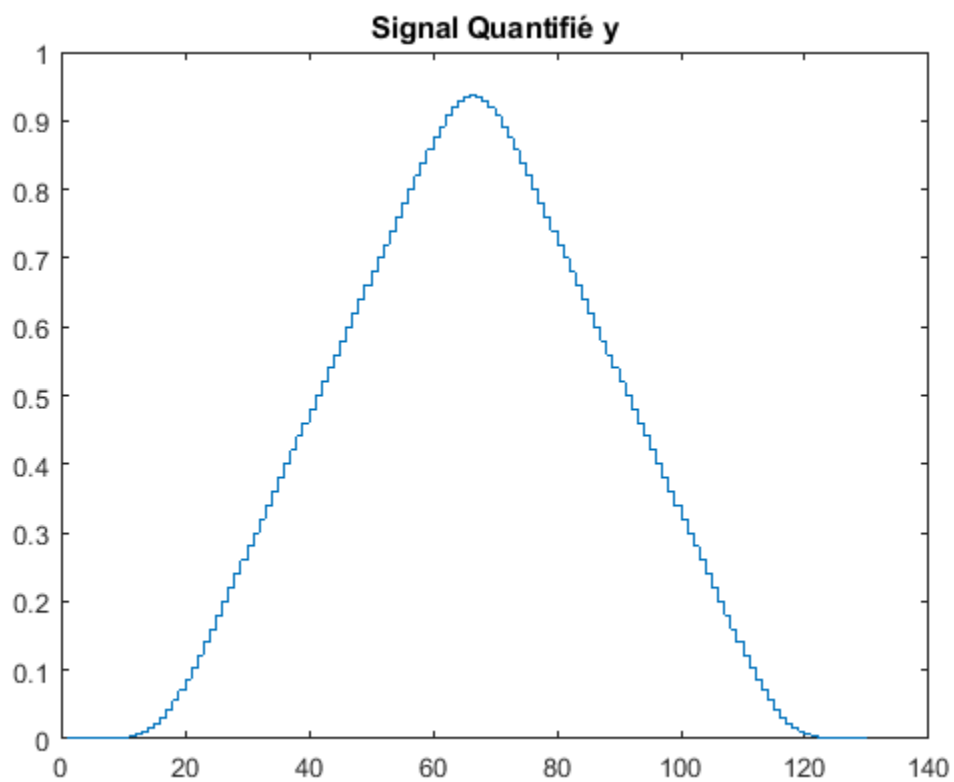
    31

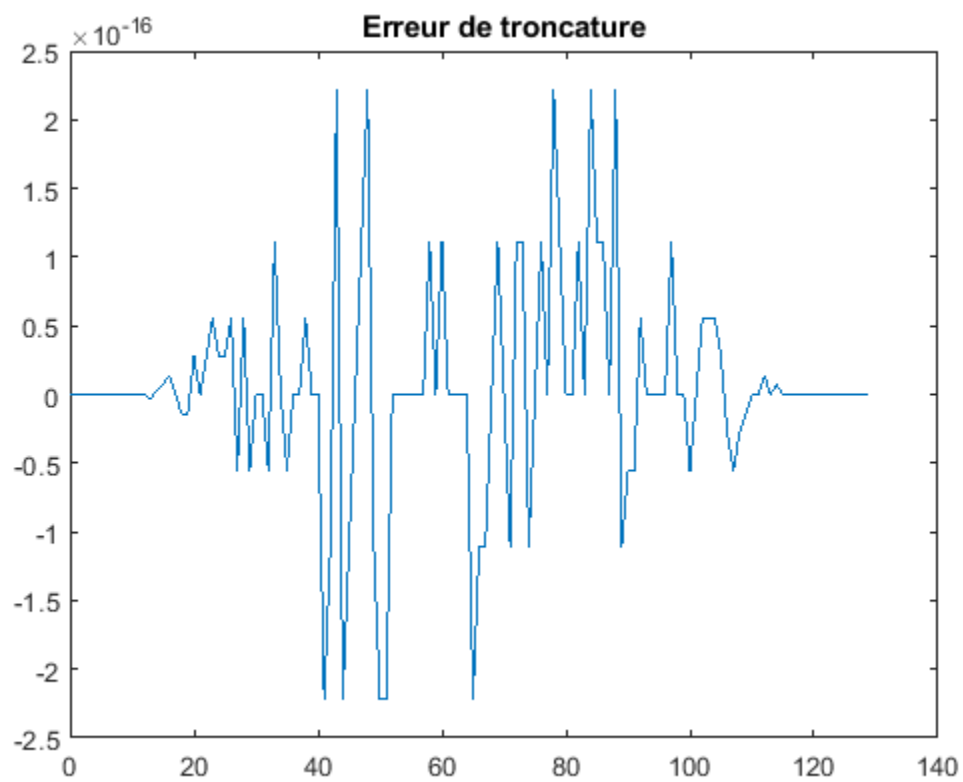
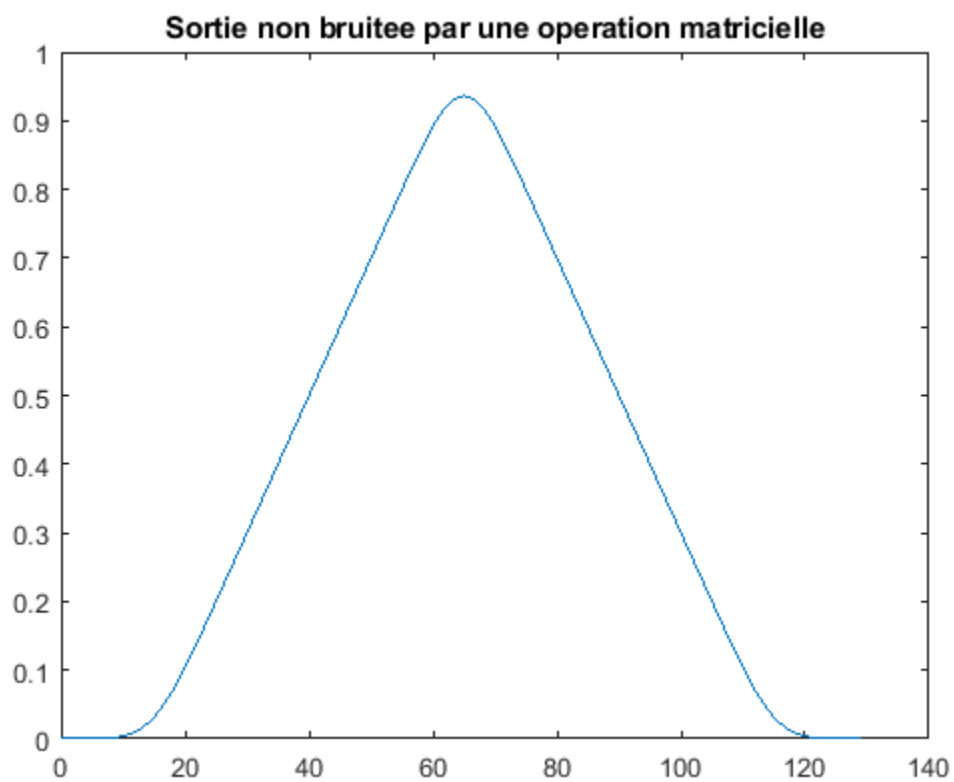
snr =

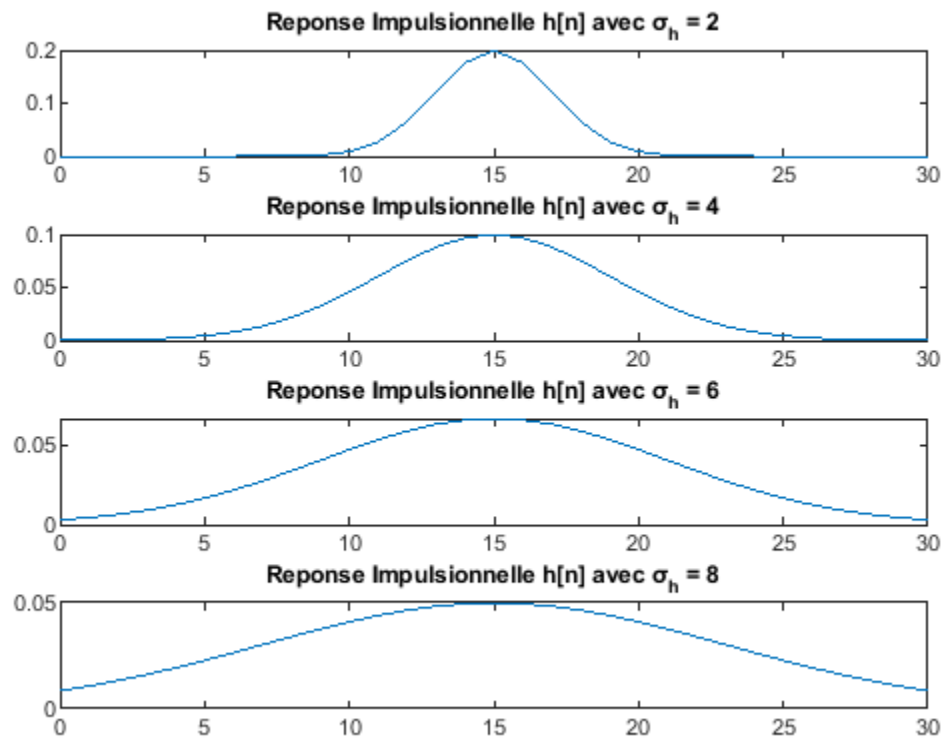
    10.9495

```



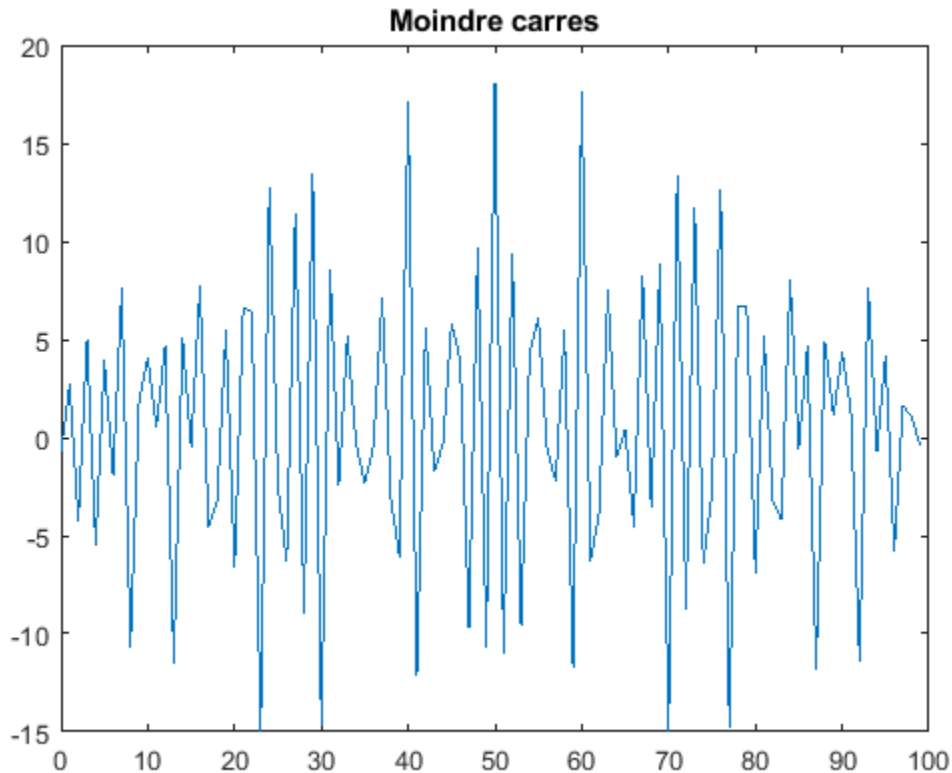






## Affichage du signal déconvolué

```
figure(9);  
plot(tx, x_mc);  
title('Moindre carres');
```



## Calcul de l'énergie de l'erreur de reconstruction

```
Erreur_e = x - x_mc';
Pe = sum(sum(Erreur_e.^2));
```

```
[U, S, V] = svd(transpose(H)*H);
figure(10)
plot(diag(S));
title('Valeurs singulières');
```

```
Conv= cond(transpose(H)*H);
```

%Par rapport la question avant, on sait bien que la valeur du  $\sigma_h$  été bien augmentée. La durée de la réponse impulsionnelle doit devient plus grande. Donc pour bien conditionner avec le filtre il faut que la valeur du  $\sigma_h$  était bien adapter la durée de la réponse impulsionnelle. On peut voir les figures de la réponse impulsionnelle avec les valeurs du  $\sigma_h$  différentes. Trouver une meilleure valeur  $\sigma_h$ .

```
% Construction de la matrice carrée D1
```

---

```

dcoll1 = zeros(1, Nx);
dcoll1(1:2) = dcoll1(1:2) + [1 -1];
dlig1 = zeros(1, Nx);
dlig1(1,1) = dcoll1(1,1);
D1 = toeplitz(dcoll1, dlig1);

% Construction de la matrice carrée D2 avec le remplacement
dcoll2 = zeros(1, Nx);
dcoll2(1:3) = dcoll2(1:3) + [1 -2 1];
dlig2 = zeros(1, Nx);
dlig2(1,1) = dcoll2(1,1);
D2 = toeplitz(dcoll2, dlig2);

% Définition des différents paramètres alpha
alpha_1 = -100;
alpha_2 = -10;
alpha_3 = 0;
alpha_4 = 10;
alpha_5 = 100;

% Calcul des solutions régularisées pour chaque valeur de alpha
x_chapeau_1 = inv(H'*H + alpha_1*D1'*D1) * H'*y';
figure(11);
subplot(5,1,1);
plot(tx, x_chapeau_1);
title('alpha1 = -100');
x_chapeau_2 = inv(H'*H + alpha_2*D1'*D1) * H'*y';
subplot(5,1,2);
plot(tx, x_chapeau_2);
title('alpha2 = -10');
x_chapeau_3 = inv(H'*H + alpha_3*D1'*D1) * H'*y';
subplot(5,1,3);
plot(tx, x_chapeau_3);
title('alpha3 = 0');
x_chapeau_4 = inv(H'*H + alpha_4*D1'*D1) * H'*y';
subplot(5,1,4);
plot(tx, x_chapeau_4);
title('alpha4 = 10');
x_chapeau_5 = inv(H'*H + alpha_5*D1'*D1) * H'*y';

% Affichage des résultats

subplot(5,1,5);
plot(tx, x_chapeau_5);
title('alpha5 = 100');

% Définition de l'intervalle des valeurs de alpha
%%Calculer et Tracer l'évolution de l'énergie de l'erreur de reconstruction

```

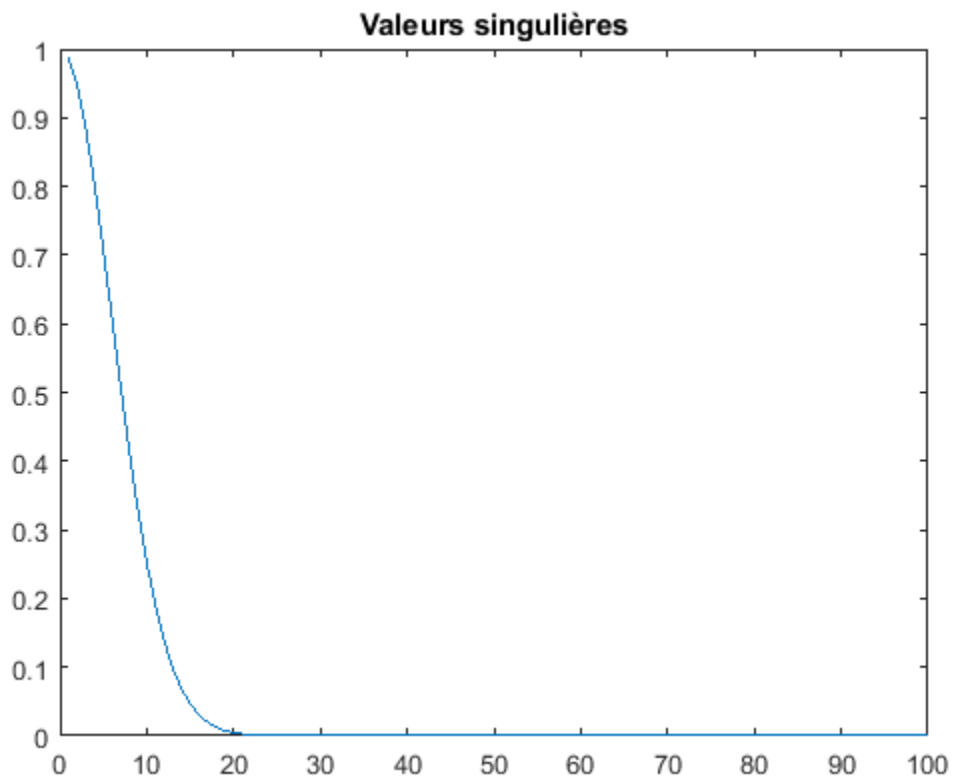
---

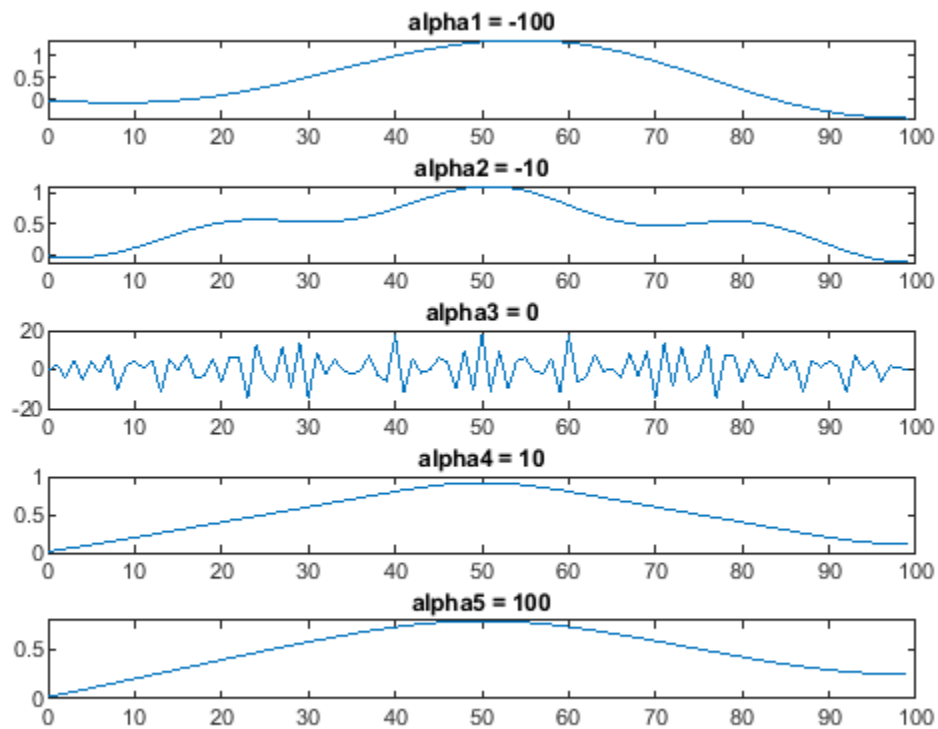
---

```
en fonction de  $\alpha$ 
minalpha = -7;
pasalpha = 0.1;
maxalpha = 2;

% Initialisation du vecteur pour stocker l'énergie de l'erreur
P_Rec = zeros(1, (maxalpha - minalpha)/pasalpha + 1);
ialpha = 0;

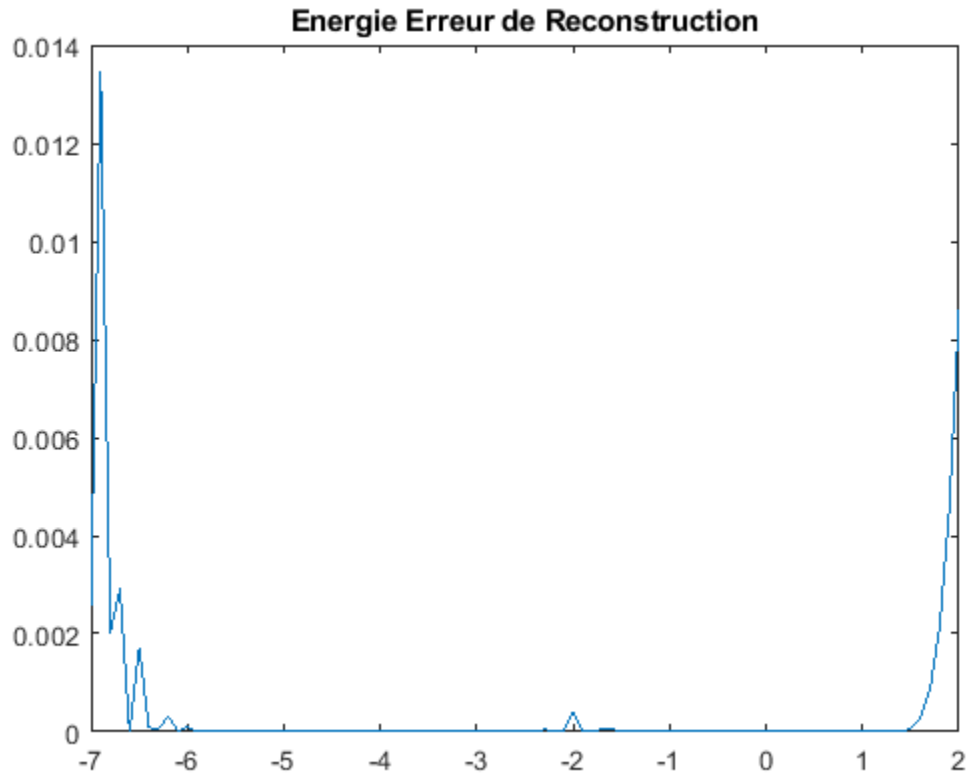
% Boucle pour calculer l'erreur pour chaque valeur de alpha
for varalpha = minalpha:pasalpha:maxalpha
    ialpha = ialpha + 1;
    alpha = 10^varalpha;
    x_chapeau = inv(H'*H + alpha*D1'*D1) * H'*y';
    Erreur_Rec = x_chapeau - x';
    P_Rec(ialpha) = sum(sum(Erreur_Rec(ialpha).^2));
end
```





## Tracé de l'énergie de l'erreur en fonction de alpha

```
figure(12);  
t_alpha = minalpha:pasalpha:maxalpha;  
plot(t_alpha, P_Rec);  
title('Energie Erreur de Reconstruction');  
%Et puis trouver une valeur minimise l'énergie de l'erreur de reconstruction
```



## Recherche de la valeur de alpha qui minimise l'énergie de l'erreur

```
P_Rec_min = min(P_Rec);  
display(P_Rec_min);
```

```
P_Rec_min =  
  
3.6156e-10
```

*Published with MATLAB® R2024a*